ClarusLang

Formal Language Specification

A Minimal Imperative Programming Language for Compiler Construction

Prepared for: Compiler Construction Course Project

Version 1.0

2. Data Types

ClarusLang supports four primitive data types. No composite, derived, or user-defined types are permitted.

Keyword	Meaning	Size (Bytes)
int	Integer	2
float	Floating Point	4
char	Character	1
string	Sequence of Characters	variable

2.1 Lexeme Table

DT denotes the token class Data Type.

Lexeme	Class-Part / Token
int	DT
float	DT
char	DT
string	DT

3. Variable Declaration

3.1 Single Variable Declaration

Syntax: DT ID ;

```
int a;
```

Regex:

ID ⟶ (L | _)(L | D | _)*

DT ⟶ int | float | char | string

DT ID ;

Explanation: a data-type keyword, followed by one or more spaces, a valid identifier, and a terminating semicolon.

3.2 Multiple Variable Declaration

Syntax: DT ID (, ID)* ;

```
int a, b, c;
```

Regex:

ID ⟶ (L | _)(L | D | _)*

DT ⟶ int | float | char | string

DT ID (, ID)* ;

3.3 Lexeme Table

Lexeme	Token
int	DT
a	ID
,	COMMA
b	ID

Lexeme	Token
c	ID
;	TERMINATOR

4. Variable Initialization

Syntax: DT ID = LITERAL ;

```
int a = 5;
float pi = 3.14;
char grade = 'A';
string name = "Hadi";
```

Regex (general form): DT ID = (IntLiteral | FloatLiteral | CharLiteral | StringLiteral) ;

Explanation: the data type and identifier are declared exactly as in Section 3, followed by an assignment operator and a literal value matching the declared type's literal grammar (Section 5).

4.1 Lexeme Table

Lexeme	Token
int	DT
a	ID
=	ASSIGN_OP
5	NUM
;	TERMINATOR
float	DT
pi	ID
3.14	FLOAT_LIT
char	DT
grade	ID
'	SQUOTE
A	CHAR_LIT
string	DT
name	ID
"	DQUOTE
Hadi	STRING_LIT

5. Constants / Literals

Symbol	Meaning
L	Alphabet (`A-Z`, `a-z`)
D	Digit (`0-9`)
ε	Empty string

5.1 Integer Literal

Regex: (+ | - | ε) D+

Examples: 45, -18, 13580

5.2 Float Literal

Regex: (+ | - | ε) D+ . D+

Examples: 3.14, -0.5, 100.25

5.3 Character Literal

Regex: '' (L | D) '

Examples: 'A', 'z'

5.4 String Literal

Regex: " (L | D | SpecialCharacter)* "

Examples: "Karachi", "Hello World"

5.5 Lexeme Table

Lexeme	Token
45	NUM
-18	NUM
3.14	FLOAT_LIT
-0.5	FLOAT_LIT
'A'	CHAR_LIT
"Karachi"	STRING_LIT

6. Variable Naming Rules (Identifiers)

- Must begin with a letter (a-z, A-Z) or an underscore (_).
- Remaining characters may contain letters, digits, or underscores.
- Cannot begin with a digit.
- Reserved keywords (Section 15) cannot be used as identifiers.
- Identifiers are case-sensitive.

Regex: ID ⟶ (L | _) (L | D | _)*

Examples: total, _count, name1, studentAge

6.1 Lexeme Table

Lexeme	Token
total	ID
_count	ID
name1	ID
studentAge	ID

7. Comments (no need for this in lexical analyzer) no tokens are formed

7.1 Single-Line Comment

```
// this is a single-line comment
```

Regex: // Character*

7.2 Multi-Line Comment

```
/* this is a
   multi-line comment */
```

Regex: /* Character* */

Lexeme	Token

8. Loop

ClarusLang provides exactly one iterative construct, the for loop, chosen for its self-contained initialization/condition/update structure, which simplifies lexical and syntax analysis.

8.1 Components

Component	Purpose
for	Reserved keyword marking the start of the loop construct.
Initialization	Declares and initializes the loop counter, executed once.
Condition	A boolean expression evaluated before every iteration.
Counter Update	An increment/decrement expression executed after every iteration.
Body	The block of statements enclosed in { } executed while the condition is true.

8.2 Syntax

```
for ( DT ID = NUM ; ID relop NUM ; ID++ )
{
    statements
}
```

8.3 Example

```
for (int i = 0; i < 10; i++)
{
    print(i);
}
```

Regex :

Initialization ⟶ DT ID = IntLiteral

Condition ⟶ ID RelOp Literal

Counter ⟶ ID (++ | --)

for (Initialization ; Condition ; Counter) {

Body}

8.4 Lexeme Table

Lexeme	Token
for	FOR
(	LPAREN

Lexeme	Token
int	DT
i	ID
=	ASSIGN_OP
0	NUM
;	TERMINATOR
i	ID
<	LT_OP
10	NUM
;	TERMINATOR
i	ID
++	INC_OP
)	RPAREN
{	LBRACE
print	PRINT
(	LPAREN
i	ID
)	RPAREN
;	TERMINATOR
}	RBRACE

9. Conditional Statements

9.1 if

```
if (condition)
{
    statements
}
```

Lexeme	Token
if	IF
(	LPAREN
condition	expr
)	RPAREN
{	LBRACE
}	RBRACE

9.2 if-else

```
if (condition)
{
    statements
}
else
{
    statements
}
```

Lexeme	Token
if	IF
else	ELSE
(	LPAREN
)	RPAREN
{	LBRACE
}	RBRACE

9.3 switch

```
switch (value)
{
    case 1:
        statements
        break;
    default:
        statements
}
```

Lexeme	Token
switch	SWITCH
case	CASE
default	DEFAULT
break	BREAK
:	COLON
(	LPAREN
)	RPAREN
{	LBRACE
}	RBRACE

10. Operators

10.1 Categories

Category	Operators
Arithmetic	+ - * / %
Relational	< > <= >=
Equality	== !=
Logical	&& || !
Assignment	=
Increment / Decrement	++ --

10.2 Lexeme Table

Lexeme	Token
+	ADD_OP
-	SUB_OP
*	MUL_OP
/	DIV_OP

Lexeme	Token
%	MOD_OP
<	LT_OP
>	GT_OP
<=	LE_OP
>=	GE_OP
==	EQ_OP
!=	NEQ_OP
&&	AND_OP
||	OR_OP
!	NOT_OP
=	ASSIGN_OP
++	INC_OP
--	DEC_OP

11. Special Characters

Lexeme	Token
(	LPAREN
)	RPAREN
{	LBRACE
}	RBRACE
[	LBRACKET
]	RBRACKET
;	TERMINATOR
,	COMMA
:	COLON
'	SQUOTE
"	DQUOTE

Note: square brackets [ ] are reserved special characters in the token set but have no active grammar production in this version of ClarusLang, since array types are intentionally excluded.

12. Program Start and End

Every ClarusLang program has exactly one entry point.

```
#activate

 statements

#deactivate
```

Execution begins at the opening brace of `activate()` and terminates at deactivate(). No statement may appear outside this block.

12.1 Lexeme Table

Lexeme	Token
activate	MAIN
(	LPAREN
)	RPAREN
deactivate	END

13. Input Statement

Syntax: input (ID) ;

```
input(age);
```

Regex: input (ID) ;

13.1 Lexeme Table

Lexeme	Token
input	INPUT
(	LPAREN
age	ID
)	RPAREN
;	TERMINATOR

14. Output Statement

Syntax: print (expression) ;

```
print("Hello");
print(a);
```

Regex: print (Expression) ;

Where Expression → ID | LITERAL

14.1 Lexeme Table

Lexeme	Token
print	PRINT
(	LPAREN
"	DQUOTE
Hello	STRING_LIT
"	DQUOTE
)	RPAREN
;	TERMINATOR
a	ID

15. Reserved Keywords

The following table lists every reserved keyword in ClarusLang. Reserved keywords may not be used as identifiers.

Keyword	Token
activate	MAIN
deactivate	END
int	DT
float	DT
char	DT
string	DT
if	IF
else	ELSE
switch	SWITCH
case	CASE
default	DEFAULT
for	FOR
break	BREAK
continue	CONTINUE
input	INPUT
print	PRINT

16. Complete Token Set (Master Table)

The master table below consolidates every lexeme class defined in this specification into the complete token set recognized by the ClarusLang lexical analyzer.

Lexeme / Pattern	Token
`#activate`	MAIN
`#deactivate`	END
`int`	DT
`float`	DT
`char`	DT
`string`	DT
`if`	IF
`else`	ELSE
`switch`	SWITCH
`case`	CASE
`default`	DEFAULT
`for`	FOR
`break`	BREAK
`continue`	CONTINUE
`input`	INPUT
`print`	PRINT
`(L | _)(L | D | _)*`	ID
`(+ | - | ε) D+`	NUM
`(+ | - | ε) D+ . D+`	FLOAT_LIT

Lexeme / Pattern	Token
' L '	CHAR_LIT
" (L | D | SpecialCharacter)* "	STRING_LIT
+	ADD_OP
-	SUB_OP
*	MUL_OP
/	DIV_OP
%	MOD_OP
<	LT_OP
>	GT_OP
<=	LE_OP
>=	GE_OP
==	EQ_OP
!=	NEQ_OP
&&	AND_OP
||	OR_OP
!	NOT_OP
=	ASSIGN_OP
++	INC_OP
--	DEC_OP
(	LPAREN
)	RPAREN
{	LBRACE
}	RBRACE
[	LBRACKET
]	RBRACKET
;	TERMINATOR
,	COMMA
:	COLON
'	SQUOTE
"	DQUOTE

Appendix A: Sample Program

The following complete program demonstrates the constructs defined in this specification.

```
#activate

    int total;
    int i;
    total = 0;
    input(total);

    for (int i = 0; i < 10; i++)
    {
        if (i % 2 == 0)
```

```
        {
            print(i);
        }
        else
        {
            continue;
        }
    }
#deactivate
```